Documento de Arquitetura (v1.0)

AES Top-Level System – Trilha RTL

Equipe de Desenvolvimento:
Daniel Lima Neto

27 de outubro de 2026

1 Visão Geral e Escopo do Projeto

O presente documento constitui a especificação arquitetural formal (versão 1.0) do subsistema acelerador criptográfico **AES Top-Level System**, desenvolvido no âmbito da trilha de projeto digital RTL. O circuito implementa um processador criptográfico baseado no padrão internacional *Advanced Encryption Standard* (AES, especificado pela norma NIST FIPS 197), operando integrado a um sistema de topo gerenciado por uma interface serial periférica SPI (*Serial Peripheral Interface*), com canal de alta largura de banda para transferência de blocos com uma memória SRAM externa e módulos integrados de geração de relógio (PLL) e condicionamento de *reset*.

Esta arquitetura foi concebida para atender rigorosamente à padronização e paridade estabelecidas no projeto integrador da trilha parceira de *Design Verification* (DV).

O entregável referente à **Semana 2** compreende este Documento de Arquitetura (v1.0), englobando o diagrama de blocos, detalhamento exaustivo das interfaces de topo e internas, modelos comportamentais do PLL e da SRAM, mapa de registradores mapeados em memória e a fundamentação técnica das decisões arquiteturais para a sabatina oral de defesa técnica.

2 Arquitetura de Topo e Diagrama de Blocos

O sistema opera segmentado em dois domínios de relógio com características temporais independentes e assíncronas entre si:

- **Domínio Serial `SCLK`:** Provido externamente pelo mestre SPI, alimentando exclusivamente a interface serial periférica (*shift register*, contador de bits e linha `spi_miso`).
- **Domínio do Sistema `sys_clk`:** Sintetizado internamente pelo multiplicador PLL a partir de um oscilador de referência externo estável (`clk_osc`), alimentando o banco de registradores de configuração e status, o subsistema de controle e datapath do núcleo AES e o controlador da memória SRAM.

A Figura 1 ilustra a arquitetura global do circuito integrado (*TOP SYSTEM*), evidenciando os sinais de fronteira, a camada de isolamento CDC, o subsistema de gerenciamento de relógio e reset e a interconexão com a memória de dados.

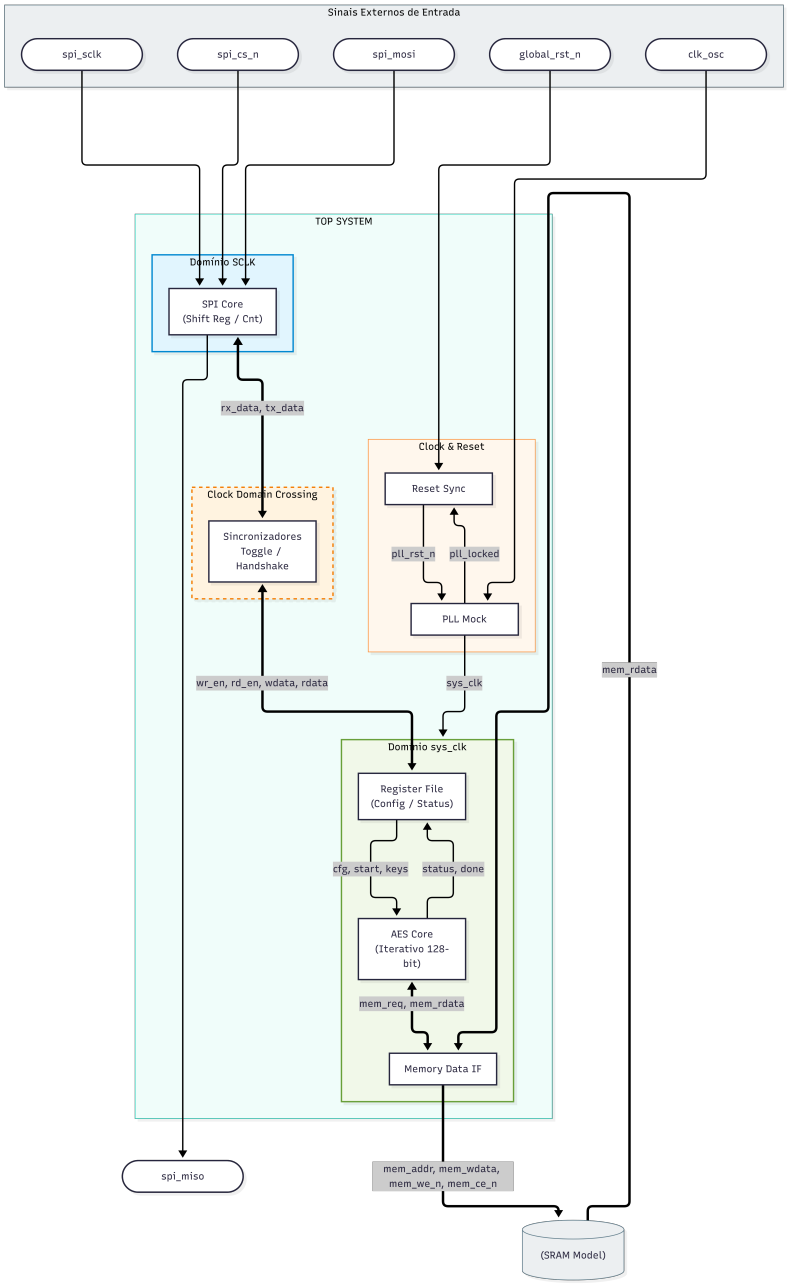


Figura 1: Diagrama de blocos da arquitetura de topo do sistema (*TOP SYSTEM*) e interfaces externas.

2.1 Descrição dos Módulos Principais

Conforme delineado no diagrama arquitetural da Figura 1, o sistema é composto por seis módulos estruturais:

1. **SPI Core (Front-End Serial no Domínio `SCLK`):**
   Executa a recepção dos quadros de comunicação seriais operando em Modo 0 (CPOL = 0, CPHA = 0). Amostra a linha `spi_mosi` nas bordas de subida de `spi_sclk` e aciona a linha de saída `spi_miso` nas bordas de descida. Contém o registrador de deslocamento de 48/56 bits e o contador modular de bits, operando de forma autônoma sob o relógio externo provido pelo mestre.

2. **Clock Domain Crossing – CDC (Camada de Fronteira):**
   Isola os domínios assíncronos `SCLK` e `sys_clk`, implementando sincronizadores baseados em transição de nível (*Toggle Synchronizers*) com barramentos de dados paralelos estáticos. Garante imunidade contra falhas de metaestabilidade e perda de pulsos de escrita ou leitura sem o elevado consumo de área de FIFOs assíncronas complexas.

3. **Clock & Reset Management (Sincronizador de Reset e PLL Mock):**
   Conjunto composto pelo modelo comportamental do sintetizador analógico de clock (`pll_mock`) e pelo sincronizador de reset de dois estágios (*Reset Synchronizer*). Controla as árvores de desasserção síncrona dos resets `pll_rst_n`, `sys_rst_n` e `spi_rst_n`.

4. **Register File (Banco de Registradores no Domínio `sys_clk`):**
   Banco de registradores mapeados em palavras de 32 bits acessíveis via SPI. Armazena os parâmetros de controle (`CTRL`), vetor de chave criptográfica de 128 bits (`KEY_0` a `KEY_3`), endereçamento e comprimento de blocos (`MEM_CFG`, `BLOCK_LEN`), parâmetros de multiplicação de clock (`PLL_CFG`) e disponibiliza o estado do sistema (`STATUS`).

5. **AES Core (Núcleo Acelerador Rijndael no Domínio `sys_clk`):**
   Processador de computação criptográfica com datapath nativo de 128 bits e arquitetura iterativa de 1 ciclo por rodada. Executa as transformações de *SubBytes*, *ShiftRows*, *MixColumns* e *AddRoundKey* (e suas contrapartes inversas na decifragem) em 10 rodadas para chaves de 128 bits.

6. **Memory Data Interface (Controlador Mestre de SRAM no Domínio `sys_clk`):**
   Controlador de memória que atua como barramento mestre para a SRAM externa (`sram_model`). Traduz requisições de blocos de 128 bits oriundas do `aes_core` em sequências síncronas de leitura e escrita conforme a largura física configurada na memória.

2.2 Detalhamento das Interfaces de Topo (Pinout)

A Tabela 1 apresenta a especificação completa e rigorosa de todas as portas externas de topo do chip (*Pinout*), categorizadas em conformidade com as quatro interfaces canônicas do sistema.

Tabela 1: Detalhamento das interfaces externas de topo (*Pinout*).

Grupo	Sinal	Direção	Largura	Domínio	Descrição Funcional
GLOBAL RESET	`global_rst_n`	Input	1 bit	Assíncrono	Reset mestre externo assíncrono, ativo em nível baixo (*active-low*). Reinicializa o circuito e o PLL.
clk from OSC	`clk_osc`	Input	1 bit	`clk_osc`	Relógio de referência do oscilador externo (frequência de referência, ex.: 10 a 50 MHz).
SPI IF	`spi_sclk`	Input	1 bit	`SCLK`	Relógio serial fornecido pelo mestre SPI externo.
	`spi_cs_n`	Input	1 bit	`SCLK`	Chip Select ativo em nível baixo. Delimita janelas de transmissão serial.
	`spi_mosi`	Input	1 bit	`SCLK`	Master Out Slave In. Entrada serial de comandos, endereços e dados.
	`spi_miso`	Output	1 bit	`SCLK`	Master In Slave Out. Saída serial de dados lidos enviada ao mestre.
MEMORY DATA IF	`mem_clk`	Output	1 bit	`sys_clk`	Relógio síncrono fornecido à memória SRAM externa.
	`mem_ce_n`	Output	1 bit	`sys_clk`	Chip Enable da memória SRAM (ativo em nível baixo).
	`mem_we_n`	Output	1 bit	`sys_clk`	Write Enable da SRAM (0 = escrita, 1 = leitura).
	`mem_addr`	Output	16 bits	`sys_clk`	Barramento de endereçamento de palavras na SRAM externa.
	`mem_wdata`	Output	32/128 b	`sys_clk`	Barramento de dados de escrita enviados para a SRAM.
	`mem_rdata`	Input	32/128 b	`sys_clk`	Barramento de dados de leitura retornados pela SRAM.

3 Interfaces Internas entre os Módulos

A Tabela 2 consolida os contratos de sinalização interna entre os blocos constituintes do *Top System*. A explicitação desses barramentos estabelece fronteiras nítidas para o desenvolvimento modular em RTL e para a instrumentação do ambiente de verificação parceiro em UVM.

Tabela 2: Contratos das interfaces internas entre módulos do sistema.

ID	Origem → Destino	Sinais e Larguras	Domínio	Protocolo / Mecanismo
I1	`spi_core` → `cdc_sync`	`rx_rw` (1), `rx_addr` (8), `rx_wdata` (32), `wr_req_toggle` (1), `rd_req_toggle` (1)	`SCLK`	Barramentos estáticos mantidos estáveis após recepção do quadro; transição *toggle* sinaliza requisição.
I2	`cdc_sync` → `spi_core`	`tx_rdata` (32), `rd_ack_toggle` (1)	`SCLK`	Dado lido estabilizado no domínio `SCLK` antes do início dos ciclos de transmissão no pino `spi_miso`.
I3	`cdc_sync` → `regbank`	`reg_wr_en` (1), `reg_rd_en` (1), `reg_addr` (8), `reg_wdata` (32)	`sys_clk`	Pulsos síncronos de exatamente 1 ciclo gerados por detector de borda XOR sobre sinal sincronizado em 2 FFs.
I4	`regbank` → `cdc_sync`	`reg_rdata` (32), `reg_rd_ack` (1)	`sys_clk`	Dado de leitura disponibilizado de forma combinacional ou em 1 ciclo e confirmado por pulso de *ack*.
I5	`regbank` → `aes_core`	`start_i` (1), `enc_dec_i` (1), `key_i` (128), `src_addr_i` (16), `dst_addr_i` (16), `num_blocks_i` (16)	`sys_clk`	`start_i` é pulso de 1 ciclo disparado pela escrita em `CTRL[0]`; parâmetros permanecem estáticos na execução.
I6	`aes_core` → `regbank`	`busy_o` (1), `done_o` (1), `err_o` (1)	`sys_clk`	`busy_o` ativo em nível durante a computação; `done_o` e `err_o` sinalizam término no registrador `STATUS`.
I7	`regbank` → `pll_mock`	`cfg_mult_i` (4), `bypass_i` (1)	`sys_clk`	Conexão direta dos bits do registrador `PLL_CFG` (resetado exclusivamente por `global_rst_n`).
I8	`pll_mock` → `Reset Sync`, `regbank`	`clk_out` (1) [= `sys_clk`], `locked_o` (1) [= `pll_locked`]	`sys_clk`	Relógio do sistema sintetizado e sinal atestando travamento de fase e estabilidade de frequência.
I9	`Reset Sync` → Módulos	`pll_rst_n` (1), `sys_rst_n` (1), `spi_rst_n` (1)	Vários	Árvores de reset com liberação síncrona em seus respectivos domínios de relógio.
I10	`aes_core` ↔ `mem_data_if`	`mem_ce_no` (1), `mem_we_no` (1), `mem_addr_o` (16), `mem_wdata_o` (128), `mem_rdata_i` (128)	`sys_clk`	Interface mestre padrão com barramento de dados nativo de 128 bits para o núcleo criptográfico.
I11	`mem_data_if` ↔ SRAM	Pinos de topo descritos no grupo **MEMORY DATA IF**	`sys_clk`	Executa os ciclos de leitura/escrita na SRAM externa;

4 Estratégia de Clock, Reset e CDC (Clock Domain Crossing)

4.1 Distribuição de Domínios de Clock

O sistema adota uma divisão estrita entre dois domínios assíncronos:

1. **Domínio Serial `SCLK`:** Opera diretamente a partir do pino `spi_sclk` fornecido pelo mestre SPI externo. O registrador de deslocamento de recepção/transmissão, o contador de bits e o driver de saída do pino `spi_miso` residem integralmente neste domínio. Isso garante que o periférico SPI funcione corretamente em qualquer velocidade de transmissão arbitrária (desde frequências muito baixas até o limite de temporização $f_{SCLK} \leq f_{sys}/2$), sem exigir sobreamostragem ou amostragem síncrona que desperdiçaria potência e dependeria de relógio interno veloz.

2. **Domínio do Sistema `sys_clk`:** Sintetizado internamente pelo multiplicador PLL a partir de `clk_osc`. Todos os blocos computacionais de alta complexidade (Banco de Registradores, Máquina de Estados, Datapath Rijndael do AES e interface com a memória SRAM) rodam sob `sys_clk`.

4.2 Comportamento Autônomo e Inicialização do PLL Mockado

O módulo `pll_mock` atua como gerador central de relógio para o sistema:

- **Inicialização Autônoma no Reset:** Ao sair do estado de reset (`global_rst_n` vai para nível alto), o modelo do PLL inicia sua oscilação com parâmetros pré-definidos padrão (*default*: fator de multiplicação $N = 1$, sem bypass) e **trava sozinho sem qualquer comando prévio via SPI**.

- **Sinal `locked_o`:** Após a contagem determinística do número parametrizado de ciclos de referência estáveis (`LOCK_CYCLES`), o PLL assere o sinal `locked_o` = 1. Este sinal alimenta o circuito gerador de reset do sistema e é espelhado no bit `STATUS[2]`.

- **Regra de Reprogramação após Locked:** Os parâmetros de configuração do PLL residem no registrador `PLL_CFG` no banco de registradores (domínio `sys_clk`). A especificação do projeto estipula formalmente que o mestre SPI **só deve reprogramar o PLL após constatar que o sistema completou a inicialização autônoma e atingiu o estado *locked*** (`STATUS[2]` == 1).

4.3 Árvore de Reset e Sequência de Liberação

Para assegurar que nenhuma porção do circuito opere sob relógio instável ou transiente metaestável, o subsistema de reset implementa três árvores de propagação, modeladas na Figura 2.

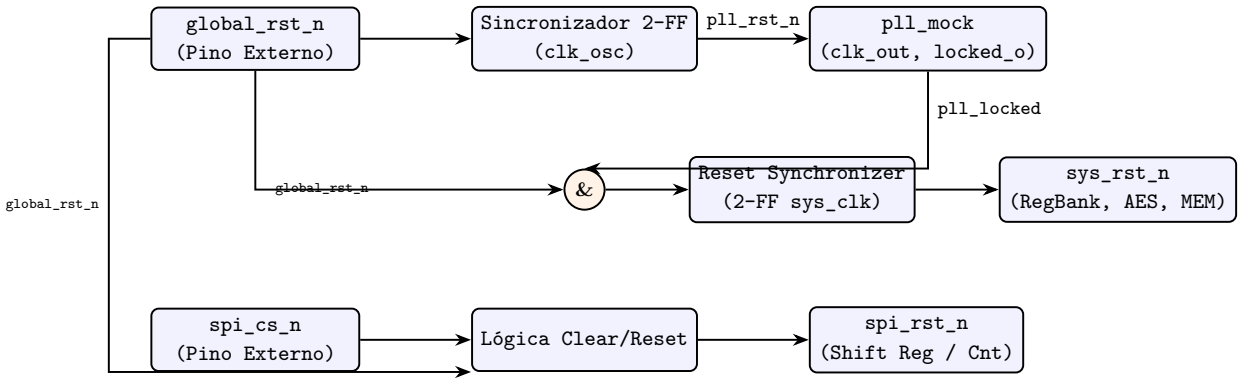


Figura 2: Diagrama esquemático estrutural das árvores de reset do sistema.

1. **Reset do PLL (`pll_rst_n`):** Derivado assincronamente de `global_rst_n` e sincronizado de forma assíncrona/síncrona na borda de `clk_osc`. Ao ser desassertado, o PLL oscila de forma autônoma até estabilizar e levar `pll_locked` = 1.

2. **Reset do Sistema (`sys_rst_n`):** Produzido por um sincronizador clássico *two-flip-flop* alimentado pelo clock sintetizado `sys_clk`. A liberação síncrona do reset do sistema obedece rigorosamente à condição:

$$\texttt{sys_rst_n} = 1 \quad \Longleftrightarrow \quad (\texttt{global_rst_n} == 1) \ \wedge \ (\texttt{pll_locked} == 1) \tag{1}$$

   Dessa forma, o banco de registradores e o datapath do AES jamais recebem ciclos de relógio antes de o sinal `pll_locked` certificar a estabilidade de fase e frequência do sistema.

3. **Reset do Domínio SPI (`spi_rst_n`):** O registrador de deslocamento e o contador de bits do *front-end* serial são mantidos em reset quando o chip está em reset global ou quando a comunicação SPI está inativa (`spi_cs_n` em nível alto):

$$\texttt{spi_rst_n} = \texttt{global_rst_n} \ \wedge \ (\sim \texttt{spi_cs_n}) \tag{2}$$

   O acoplamento com `spi_cs_n` garante imunidade contra ruídos espúrios na linha `spi_sclk` fora das janelas válidas de transmissão e reseta o contador em caso de quadros truncados.

4.4 Cuidado Arquitetural Crítico: Reprogramação Dinâmica sem Reset Deadlock

Os bits de programação do multiplicador de clock residem no registrador `PLL_CFG` no domínio `sys_clk`. Quando o mestre SPI efetua uma reprogramação em tempo de execução gravando novos parâmetros em `PLL_CFG`, o sinal analógico `pll_locked` cai temporariamente para nível lógico baixo (0) durante o período de readaptação de fase.

Solução para Prevenção de Deadlock: Se o registrador `PLL_CFG` recebesse o reset sincronizado `sys_rst_n`, a queda de `pll_locked` forçaria o registrador a retornar instantaneamente ao seu valor de reset padrão (*default*), revertendo a configuração antes da estabilização e travando o circuito em um laço infinito de reinicialização (*reset deadlock loop*). Por esta razão técnica fundamental:

- O registrador `PLL_CFG` é conectado **exclusivamente ao sinal `global_rst_n`**;

- Todos os demais registradores e a lógica de processamento permanecem protegidos por `sys_rst_n`.

4.5 Travessia de Domínio de Clock (CDC: `SCLK` ↔ `sys_clk`)

A comunicação bidirecional entre o *front-end* serial e o núcleo síncrono adota sincronizadores de pulso baseados em chaveamento de nível lógico (***Toggle Synchronizer***) acoplados a barramentos de dados paralelos estáticos:

- **Fluxo de Escrita (`SPI` → `RegBank`):** Ao final da amostragem do 48º bit de um quadro de escrita serial, o SPI congela os barramentos paralelos de endereço e dados (`rx_addr`, `rx_wdata`) e inverte o nível lógico de um bit de requisição (`wr_req_toggle`). No domínio `sys_clk`, este bit atravessa uma cadeia de dois flip-flops sincronizadores conectada a uma porta XOR (detector de borda), gerando um pulso síncrono `reg_wr_en` com largura de exatamente 1 ciclo de `sys_clk`, que realiza a escrita atômica no banco de registradores.
- **Fluxo de Leitura (`RegBank` → `SPI`):** Adota-se protocolo com ***Dummy Byte*** de 8 ciclos de relógio serial em `SCLK`. O mestre SPI envia o comando de leitura e o endereço de destino (16 ciclos) e transmite 8 ciclos de *dummy* em seguida. Esse intervalo provê uma janela determinística de tempo ($\approx 8\,T_{SCLK}$) para a travessia CDC de ida ($\approx 2\,T_{sys}$), leitura do registrador no banco ($\approx 1\,T_{sys}$) e travessia CDC do dado de retorno ao domínio `SCLK` ($\approx 2\,T_{SCLK}$), satisfazendo a temporização para qualquer relação $f_{sys} \geq f_{SCLK}$ sem necessitar de pinos de *wait-state*.

Tabela 3: Formato e anatomia dos quadros do protocolo SPI (Modo 0: CPOL = 0, CPHA = 0).

Operação	**Sequência de Linha (`MOSI` / `MISO`)**	**Ciclos de `SCLK`**
Escrita	`MOSI`: `CMD`[7:0] (`0x02`) | `ADDR`[7:0] | `WDATA`[31:0]. `MISO` permanece em alta impedância.	48 ciclos
Leitura	`MOSI`: `CMD`[7:0] (`0x03`) | `ADDR`[7:0] | *Dummy Byte* [7:0]. `MISO`: `RDATA`[31:0] após o *dummy*.	56 ciclos

5 Mapa de Registradores

O subsistema dispõe de um espaço de endereçamento alinhado em palavras de 32 bits, detalhado rigorosamente na Tabela 4.

Tabela 4: Mapa de registradores mapeados em memória (*Memory-Mapped Registers*).

Offset	Nome	Acesso	Reset	Descrição dos Campos e Bit-Fields
0x00	CTRL	R/W	0x0000_0000	**[0]** **start** (Auto-clear / Pulse): Inicia operação criptográfica. **[1]** **enc_dec**: 0 = Cifragem (Encrypt), 1 = Decifragem (Decrypt). **[2]** **irq_en**: Habilita sinalização de término no registrador de status.
0x04	STATUS	RO	0x0000_0000	**[0]** **busy**: Núcleo AES em execução (leitura, cálculo ou escrita). **[1]** **done**: Conclusão de processamento (limpo por leitura ou W1C). **[2]** **pll_locked**: Espelho direto do pino de travamento do gerador PLL. **[3]** **err**: Erro de acesso, parâmetros inconsistentes ou número de blocos nulo.
0x08	PLL_CFG	R/W	0x0000_0001	**[3:0]** **cfg_mult**: Fator multiplicador de frequência $N \in [1, 15]$. **[4]** **bypass**: Se 1, bypass do PLL (sys_clk = clk_osc). *Nota: Resetado exclusivamente por* ***global_rst_n*** *para imunidade a deadlock.*
0x0C	MEM_CFG	R/W	0x0000_0000	**[15:0]** **src_addr**: Endereço base de leitura da SRAM. **[31:16]** **dst_addr**: Endereço base de escrita do resultado na SRAM.
0x10	BLOCK_LEN	R/W	0x0000_0001	**[15:0]** **num_blocks**: Quantidade de blocos de 128 bits a processar.
0x20	KEY_0	R/W	0x0000_0000	Bits [31:0] da chave simétrica AES de 128 bits.
0x24	KEY_1	R/W	0x0000_0000	Bits [63:32] da chave simétrica AES de 128 bits.
0x28	KEY_2	R/W	0x0000_0000	Bits [95:64] da chave simétrica AES de 128 bits.
0x2C	KEY_3	R/W	0x0000_0000	Bits [127:96] da chave simétrica AES de 128 bits.

5.1 Significado dos Bits de Configuração do PLL

A Tabela 5 detalha a correspondência funcional entre os bits do registrador PLL_CFG e as portas do modelo pll_mock.

Tabela 5: Mapeamento dos sinais de configuração do `pll_mock`.

Porta	Largura	Origem	Semântica Operacional
`cfg_mult_i`	4 bits	`PLL_CFG[3:0]`	Fator de multiplicação de frequência N: $f_{sys} = f_{osc} \times N$. O valor `0x0` é reservado ($N = 1$). Padrão de reset: `0x1` ($f_{sys} = f_{osc}$).
`bypass_i`	1 bit	`PLL_CFG[4]`	Se assertado (1), desativa o multiplicador e direciona `clk_osc` diretamente para `clk_out`. Padrão de reset: 0.
`locked_o`	1 bit	Saída PLL	Indica estabilização do clock sintetizado. Espelhado em `STATUS[2]` e monitorado pelo gerador de reset.

5.2 Significado dos Campos de Configuração da Memória

A Tabela 6 descreve a semântica operacional dos campos de memória utilizados pelo núcleo AES e pelo controlador `mem_data_if`.

Tabela 6: Campos de configuração do subsistema de memória SRAM.

Campo	Largura	Registrador	Significado e Regras de Operação
`src_addr`	16 bits	`MEM_CFG[15:0]`	Endereço base de leitura da SRAM onde reside o primeiro bloco de 128 bits de entrada. Incrementado automaticamente em +16 bytes a cada bloco processado.
`dst_addr`	16 bits	`MEM_CFG[31:16]`	Endereço base de escrita na SRAM para gravação do resultado cifrado/decifrado. Incrementado em +16 bytes por bloco.
`num_blocks`	16 bits	`BLOCK_LEN[15:0]`	Número total de blocos de 128 bits a serem computados. Valor padrão de reset: `0x0001`. Se configurado em `0x0000`, a FSM sinaliza erro imediato em `STATUS[3]`.

6 Microarquitetura do AES e Interface Interna

6.1 Escolha da Microarquitetura e Análise de Trade-off

Para o acelerador do projeto integrador, selecionou-se a **Microarquitetura Iterativa por Rodada com Datapath de 128 bits**, com execução de exatamente 1 rodada por ciclo de relógio. A Tabela 7 sintetiza o confronto técnico com as alternativas arquiteturais.

Tabela 7: Comparação de microarquiteturas para o núcleo AES.

Critério	**Iterativa (128-bit) – Escolhida**	**Serial 32-bit (4× S-Box)**	**Pipelined (10 estágios)**
Área (Portas Lógicas)	Média / Balanceada (16 S-Boxes para o State + 4 para o *Key Schedule*).	Muito Baixa (4 S-Boxes compartilhadas), porém com densa malha de multiplexadores.	Muito Alta (160+ S-Boxes e centenas de registradores pipeline de 128 bits).
Latência por Bloco	11 ciclos de relógio (1 ciclo inicial de *AddRoundKey* + 10 rodadas).	44 a 54 ciclos por bloco de 128 bits (gargalo de 4 ciclos para cada rodada).	10 ciclos de latência inicial, com vazão ideal de 1 bloco por ciclo de clock.
Complexidade de Controle	Baixa / Determinística. Contador de rodadas linear de 0 a 10 de fácil verificação.	Alta. Máquinas de estado aninhadas controlando bytes e colunas individualmente.	Média no núcleo, porém ineficiente face ao gargalo das interfaces de I/O externas.

Fundamentação Técnica para a Defesa Oral: A microarquitetura iterativa de 128 bits viabiliza a solução de engenharia mais eficiente:

1. **Gargalo de I/O e Desperdício de Silício na Alternativa Pipelined:** O sistema alimenta e drena dados através de uma memória SRAM externa e de uma linha SPI serial. Uma arquitetura profundamente em *pipeline* (capaz de produzir 128 bits/ciclo) ficaria constantemente ociosa (*stalled*) aguardando transferências de memória, desperdiçando a grande área de silício de mais de 160 S-Boxes.

2. **Desempenho e Cobertura na Alternativa Serial 32-bit:** A opção serial de 32 bits adiciona expressiva latência computacional (44–54 ciclos) e máquinas de estado intricadas com contadores de bytes, dificultando o fechamento de cobertura de verificação em troca de uma economia marginal de área em nós de tecnologia modernos.

6.2 FSM de Controle do Núcleo AES

O fluxo de processamento do módulo `aes_core` é orquestrado pela máquina de estados finitos ilustrada na Figura 3.

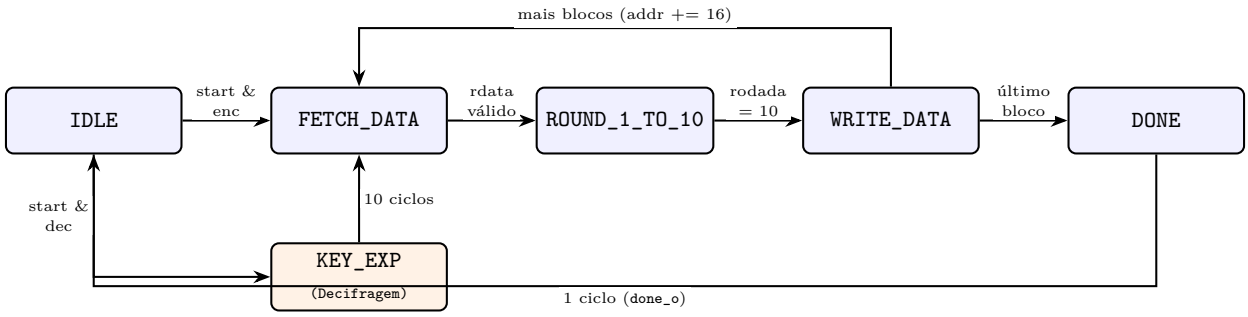


Figura 3: Diagrama de transição de estados da FSM de controle do acelerador `aes_core`.

6.3 Interface do Núcleo AES (`aes_core`) e Desenvolvimento Antecipado

Conforme o planejamento das Fases 1 e 2 do projeto, o núcleo criptográfico `aes_core` é desenvolvido e verificado na **Semana 3 de forma totalmente autônoma e prévia ao subsistema SPI**. A Listagem 1 formaliza o contrato de portas do núcleo:

Listing 1: Contrato SystemVerilog do módulo `aes_core`.

```
module aes_core (
    input  logic          clk,          // sys_clk
    input  logic          rst_n,        // sys_rst_n

    // Interface de Controle e Status (originada no Register File)
    input  logic          start_i,      // Pulso de 1 ciclo disparando a execucao
    input  logic          enc_dec_i,    // 0: Cifragem (Encrypt), 1: Decifragem
    (Decrypt)
    input  logic [127:0] key_i,         // Chave simetrica AES de 128 bits
    input  logic [15:0]  src_addr_i,    // Endereco inicial de leitura na SRAM
    input  logic [15:0]  dst_addr_i,    // Endereco inicial de escrita na SRAM
    input  logic [15:0]  num_blocks_i, // Quantidade de blocos de 128 bits a
    processar
    output logic          busy_o,       // 1 durante processamento (leitura +
    calculo + escrita)
    output logic          done_o,       // Pulso de 1 ciclo indicando termino
    dos blocos
    output logic          err_o,        // Flag de erro (ex: num_blocks == 0 ou
    timeout)

    // Interface com a Memoria de Dados (SRAM Master)
    output logic          mem_ce_no,    // Chip Enable da SRAM (ativo baixo)
    output logic          mem_we_no,    // Write Enable (0 = escrita, 1 =
    leitura)
    output logic [15:0]  mem_addr_o,   // Barramento de enderecos para a SRAM
    output logic [127:0] mem_wdata_o,  // Dado de escrita de 128 bits
    input  logic [127:0] mem_rdata_i   // Dado de leitura de 128 bits retornado
    da SRAM
);
```

Vantagem Arquitetural: Este encapsulamento permite que a equipe de RTL desenvolva um banco de testes isolado (`tb_aes_core`) acionando diretamente os vetores de teste oficiais do NIST FIPS 197 e conferindo a leitura/escrita de blocos em um modelo emulado de SRAM, antes de integrar as camadas assíncronas do SPI e CDC na Semana 4.

7 Modelos Comportamentais de Simulação (PLL e Memória)

Os modelos comportamentais de simulação (*mock models*) representam os blocos analógicos e de memória estática, assegurando compatibilidade idêntica com o ambiente adotado pela trilha parceira de Verificação (DV).

7.1 Modelo Comportamental do PLL (`pll_mock`)

O modelo do PLL recebe o sinal de referência `clk_in` e sintetiza a saída `clk_out` após a estabilização de fase governada pelo parâmetro `LOCK_CYCLES`:

Listing 2: Interface SystemVerilog do modelo comportamental `pll_mock`.

```
module pll_mock #(
    parameter int LOCK_CYCLES = 16   // Ciclos estaveis de clk_in ate locked_o
    = 1
)(
    input  logic        clk_in,      // Clock do oscilador de referencia
    (clk_osc)
    input  logic        rst_n,       // Reset ativo baixo (sincronizado em
    clk_osc)
    input  logic        bypass_i,    // Se 1, bypass ativado (clk_out = clk_in)
    input  logic [3:0] cfg_mult_i,   // Fator de multiplicacao de frequencia (N)
    output logic        clk_out,     // Clock sintetizado do sistema (sys_clk)
    output logic        locked_o     // 1 quando clock de saida estiver
    estabilizado
);
```

Comportamento em Simulação:

- Ao desassertar `rst_n`, o módulo opera no modo *default* ($N = 1$, sem bypass) e trava sozinho após 16 bordas de subida de `clk_in`, elevando `locked_o` = 1;
- Em caso de alteração em tempo de execução nos sinais `cfg_mult_i` ou `bypass_i`, `locked_o` cai para 0 no ciclo correspondente e permanece em nível baixo durante novo intervalo de acomodação de `LOCK_CYCLES` períodos de relógio.

7.2 Modelo Comportamental da Memória SRAM (`sram_model`)

A memória de dados é modelada como uma SRAM estática síncrona *Single-Port* parametrizável em profundidade e largura de dados:

Listing 3: Interface SystemVerilog parametrizada do modelo `sram_model`.

```
module sram_model #(
    parameter int ADDR_WIDTH = 10,   // Profundidade: 2^ADDR_WIDTH palavras
    parameter int DATA_WIDTH = 32    // Largura nativa da palavra (32 ou 128
    bits)
)(
    input  logic                  clk,    // Relogio sincrono (mem_clk /
    sys_clk)
    input  logic                  ce_n,   // Chip Enable (ativo baixo)
    input  logic                  we_n,   // Write Enable (0 = Escrita, 1 =
    Leitura)
    input  logic [ADDR_WIDTH-1:0] addr,   // Endereco da palavra
    input  logic [DATA_WIDTH-1:0] wdata,  // Barramento de dados de escrita
```

```
    output logic [DATA_WIDTH-1:0] rdata   // Barramento de dados lidos
    (registrado)
);
```

Protocolo de Operação:

- **Escrita Síncrona:** Quando `ce_n` = 0 e `we_n` = 0 na borda de subida de `clk`, o dado presente em `wdata` é armazenado na posição `addr`;
- **Leitura Síncrona com Latência de 1 Ciclo:** Quando `ce_n` = 0 e `we_n` = 1 na borda de subida de `clk`, a posição `addr` é acessada e disponibilizada na saída `rdata` exatamente na borda de subida seguinte.

8 Registro das Decisões de Arquitetura (Justificativa para a Defesa)

A Tabela 8 sintetiza a matriz de decisões técnicas adotadas no projeto da versão 1.0, relacionando as justificativas de engenharia e as opções descartadas para suporte à defesa técnica com o instrutor.

Tabela 8: Matriz de decisões de arquitetura e alternativas analisadas.

ID	Decisão Adotada	Justificativa Técnica	Alternativa Descartada
D1	Dois domínios nativos de clock (`SCLK` e `sys_clk`).	Isola a comunicação SPI externa de variações de frequência do núcleo, simplifica a análise de temporização estática (STA) e mitiga ruído externo.	Amostrar `SCLK` com `sys_clk`: exigiria sobreamostragem severa ($f_{sys} \gg f_{SCLK}$) e tornaria o chip vulnerável a *glitches*.
D2	CDC por *Toggle* com barramentos estáticos.	Baixo custo em área de silício e garantia matemática de integridade para dados multi-bit sem necessidade de codificação Gray.	FIFO assíncrona completa: traria overhead proibitivo de portas lógicas para transferências de registradores individuais.
D3	Leitura SPI com *Dummy Byte* (8 ciclos).	Absorve deterministicamente o atraso de sincronização de CDC sem demandar paradas dinâmicas (*wait states*) ou pinos extras de handshake.	Pino externo de *ready*/busy: aumentaria a contagem de pinos do encapsulamento (*pin count*) do chip.
D4	`PLL_CFG` resetado unicamente por `global_rst_n`.	Elimina o laço de reinicialização infinita (*reset deadlock*) quando o sinal `pll_locked` cai durante reprogramação em tempo de execução.	Conectar todo o regbank ao `sys_rst_n`: impossibilitaria alterar a frequência do circuito em operação normal.
D5	Reset do contador SPI dependente de `spi_cs_n`.	Descarta contadores truncados e garante sincronismo de byte caso o mestre interrompa um quadro no meio da transmissão.	Reset puramente global: um frame truncado desalinharia permanentemente todas as mensagens seguintes.
D6	AES Iterativo de 128 bits (1 rodada/ciclo).	Ponto ótimo de compromisso entre área, vazão computacional e esforço de verificação face ao gargalo de transferência da SRAM.	Arquitetura *pipelined* de 10 estágios (área desperdiçada) ou 32 bits (FSM excessivamente complexa).
D7	Módulo `mem_data_if` como adaptador de barramento.	Permite ao `aes_core` operar com interface canônica de 128 bits independente de a SRAM ser fisicamente de 32 ou 128 bits.	Embutir serialização de 32 bits dentro da máquina de estados do datapath do núcleo criptográfico.
D8	Registradores mapeados em palavras de 32 bits.	Alinhamento nativo com os quadros do protocolo SPI e simplicidade na escrita de chaves de 128 bits em apenas 4 acessos sequenciais.	Registradores de 8 bits: multiplicaria por 4 o número de comandos SPI para configurar chaves e endereços de bloco.